Cheers Hotel — Sales & Records App

Developer Documentation — Dual-Platform Architecture (Desktop Till + Mobile Order-Taking)

Project Owner (Client): Enose Mugalla — Cheers Hotel

Developer: Antony Arunga

Doc Version: 3.0 — July 23, 2026 (supersedes v2.0, July 23, 2026)

Change from v2.0: Reintroduced the mobile app as a first-class, day-one part of V1 rather than a deferred V2 feature. Staff can now record orders from either the desktop till or a phone; the desktop remains the only device wired to the receipt printer. Every section below (summary, mission, tech stack, printing, phases, deployment) has been updated to reflect this dual-platform split.

1. Project Summary

A POS-style app that replaces Cheers Hotel's manual receipt book. Orders can be recorded either at the desktop till or from a staff member's phone while at a table; both write to the same Firestore backend. The desktop computer at the till is the only device connected to the thermal printer, so it prints every receipt regardless of which device recorded the order. The owner gets automatic daily/weekly/monthly sales analytics, expense tracking, and profit/loss calculation — with no manual reconciliation.

This version restores the mobile app to V1 scope. The hardware purchased (Xprinter XP-Q80A, USB + LAN) and the desktop's role as the till and print engine are unchanged from v2.0 — what changes is that order-taking is no longer desktop-only. A phone app now writes orders to the same Firestore collections as the desktop, so staff can take orders tableside and the desktop till simply becomes the point where those orders get printed and reconciled.

2. Mission & Goal

Mission: give Cheers Hotel a fast, reliable, easy-to-use sales system that never loses a sale — even with poor connectivity — while giving the owner clear profit/loss visibility without lifting a finger to reconcile numbers, and letting staff take orders wherever it's fastest: at the till or at the table.

Goal: ship a lean, stable V1 (menu → order (desktop or mobile) → print (desktop) → report) that staff can learn in minutes, architected so V2 features (M-Pesa reconciliation, inventory deduction, staff logins, AI insights, true offline mode) can be added later without a rebuild.

3. Objectives

- Replace 100% of the manual receipt-book workflow with a desktop + mobile order flow.
- Zero data loss — every recorded sale must persist reliably, even with poor connectivity, from either device.
- Sub-5-second flow from "order placed" (desktop or mobile) to "receipt printed" (desktop).
- Accurate, automatic reporting with no manual reconciliation by the owner, regardless of which device recorded a given order.
- Build V1 lean and stable; architect it so V2 features can be added without a rebuild.

4. Hardware — Printer

Purchased: Xprinter XP-Q80A thermal receipt printer, from AtoZ Suppliers, Nairobi (Invoice #1559, 23/07/2026, KES 7,000 including 80mm paper roll).

Spec	Value
Model	XP-Q80A
Interface	USB + LAN (no Bluetooth)
Paper width	80mm
Print speed	230 mm/s
Command support	ESC/POS
Cash drawer port	24V, 1A
Power input	24V, 2.5A
Manufacturer	Zhuhai Xprinter Electronics Technology Co., Ltd.

The printer stays wired to the desktop only — USB is the primary connection, since it removes any dependency on the restaurant's WiFi/router being up at the moment of printing. LAN remains available as a secondary/future option. This does not change with the mobile app: orders placed on a phone are written to Firestore and printed by the desktop, exactly like desktop-recorded orders — the phone itself never talks to the printer directly.

5. Tech Stack (Revised for Dual-Platform)

Layer	Choice	Why
App Platform	Flutter — single codebase, two build targets: Windows desktop (till) and Android/iOS mobile (order-taking)	One codebase covers both; shared widgets/logic, platform-specific UI where needed (e.g. printer access only compiled into the desktop build)
Backend / DB	Firebase (Firestore + Auth)	Real-time sync, built-in offline persistence, generous free tier — same backend serves both desktop and mobile clients
Local Storage (offline-first)	Firestore offline cache (V1) on both platforms → Hive/SQLite if a fully custom offline layer is needed in V2	Firestore's offline persistence covers most of V1's needs out of the box on desktop and mobile alike
Receipt Printing	ESC/POS over USB (primary) via Windows raw print spooler (win32 API) or a USB ESC/POS Flutter/Windows package; LAN socket (port 9100) as secondary/backup path — desktop-only, regardless of which device created the order	Matches the printer actually purchased (USB+LAN, no Bluetooth); USB avoids network dependency for the critical print step
Order Sync	Mobile writes orders directly to the same Firestore orders/ collection as desktop, tagged with recordedBy and a device/source field	Desktop and mobile stay in lock-step with no separate sync layer to build
Analytics/Reports	Computed client-side from Firestore queries (V1); Cloud Functions for heavier aggregation later	Keeps V1 simple and cheap; works the same whether the underlying orders came from desktop or mobile

Hosting (future admin dashboard)	Next.js on Vercel	Optional — view sales from anywhere, added later if needed

Note: the mobile app is no longer deferred to V1.1/V2 — it ships alongside the desktop till in V1. The printer itself is still always reached from the desktop it's physically connected to; the mobile app never attempts to print.

6. Data Model (Firestore Collections)

restaurants/{restaurantId}

- menuItems/{itemId} — name, category, price, active
- orders/{orderId} — items [{itemId, name, price, qty}], total, timestamp, recordedBy, source ("desktop" | "mobile"), synced, receiptPrinted
- expenses/{expenseId} — description, amount, category, date
- dailySummaries/{date} (optional) — totalSales, totalOrders, totalExpenses, profitOrLoss, topItems

Change from v2.0: added a source field on orders so reporting and troubleshooting can distinguish desktop-recorded from mobile-recorded sales.

7. Core Features — Build Breakdown (V1)

7.1 Menu Setup

- Admin screen to add/edit/deactivate menu items (name, category, price).
- Seeded once from the client's real menu; editable anytime; changes reflect on both desktop and mobile immediately via Firestore.

7.2 Order Recording (Desktop or Mobile)

- Grid/list of active menu items, grouped by category — available on both the desktop till screen and the mobile app.
- Tap/click to add to current order, with a quantity stepper.
- "Record Sale" button (either device) → writes order to Firestore, tagged with its source.
- If recorded on the desktop, the app also triggers a local print job immediately to the USB-connected printer.
- If recorded on mobile, the order appears on the desktop app in near-real-time; the till operator confirms/prints it from there.
- Target: entire flow completable in under 10 taps/clicks for a typical order, on either device.

7.3 Receipt Printing (Desktop Only)

- Printer connects via USB directly to the till desktop — no pairing step, always available if the computer is on.
- On "Record Sale" from the desktop, or on confirming a mobile-originated order at the desktop, generate an ESC/POS formatted receipt: restaurant name, itemized list, total, date/time, optional "Thank you" footer.
- Handle printer-not-connected gracefully (queue the print job, alert the user, allow retry/reprint).

7.4 Reporting Dashboard

- Today view: total sales, order count, top 3 items — combined across desktop and mobile-sourced orders.
- Weekly view: day-by-day sales chart, week-over-week comparison.
- Monthly view: month-over-month comparison, top-selling items ranked.
- Optional breakdown by source (desktop vs. mobile) for operational visibility.

7.5 Expense Tracking

- Simple form: description, amount, category, date. Feeds directly into profit/loss. Desktop only for V1 (expenses are typically entered by the admin, not tableside staff).

7.6 Profit/Loss Calculation

profitOrLoss(period) = SUM(orders.total in period) − SUM(expenses.amount in period)

- Computed for day / week / month on demand, shown with a clear green/red indicator; unaffected by which device recorded each order.

8. Non-Functional Requirements

- Reliability: no sale should ever be lost, even if the app crashes mid-flow, on either device — write to local cache first, sync after.
- Speed: recording a sale should feel instant on both desktop and mobile; printing should feel instant during a busy service.
- Simplicity: usable by non-technical staff with minimal training — large tap/click targets, no clutter, consistent UI language across platforms.
- Data integrity: menu price changes must not retroactively alter historical order totals (store price-at-time-of-sale on the order).
- Connectivity resilience: mobile devices are more likely to be on WiFi with variable signal than the wired desktop — orders queue locally via Firestore's offline cache and sync when connectivity returns.
- Single point of failure awareness: since printing depends on one desktop, document a simple restart/recovery procedure staff can follow if the app freezes mid-service.

9. Development Phases & Milestones

Phase	Deliverable	Target
0. Setup	Firebase project, Flutter desktop + mobile scaffold, repo	Day 1–2
1. Menu Module	Add/edit/list menu items, seeded from client's real menu	
2. Order Recording	Click-to-order flow on desktop and mobile, order persistence	
3. Printer Integration	USB ESC/POS receipt printing on desktop, tested with orders from both sources	
4. Reporting	Daily/weekly/monthly views, top-sellers, comparisons	
5. Expenses & P/L	Expense entry + automatic profit/loss calc	
6. Polish & QA	Real-world testing in the actual restaurant during service, both devices in use	
7. Training & Handover	Train staff on desktop and mobile, go live	

10. Testing Plan

- Unit tests: total/price calculations, profit-loss formula, offline queue logic.
- Manual device testing: the real printer, the real desktop, at least one real staff phone, spotty WiFi/data conditions on mobile.
- Cross-device test: place simultaneous orders from desktop and mobile, confirm no lost or duplicated sales and correct print order.
- Field test: run parallel with the paper receipt book for 2–3 days before fully switching over, to catch discrepancies.

- Power-loss test: confirm no sale is lost if the desktop loses power mid-order, including any queued mobile-originated orders awaiting print.

11. Deployment Plan

- Desktop: install directly on the restaurant desktop as a Windows executable (no Play Store / app store involved).
- Mobile: install as a direct APK (Android) for V1 to avoid app-store review delays; Play Store / TestFlight distribution can follow later if useful.
- Printer driver installed once from the included CD, or configured directly if the app talks to it via raw USB printing.
- Firebase project on the Blaze (pay-as-you-go) plan — realistically stays within free-tier limits at this scale, even with two client platforms.

12. Risks & Mitigations

Risk	Mitigation
Desktop is the single point of failure for printing	Keep a backup receipt book on hand for the first weeks; document a quick restart procedure
Mobile orders arrive while desktop app is closed/asleep	Orders persist in Firestore regardless; desktop shows a pending queue on next open; consider a simple notification/sound
Staff resistance to change from paper book, or confusion using two devices	Run parallel testing period + hands-on training on both desktop and mobile before full cutover
Client wants menu price changes mid-month	Store price-at-time-of-sale on each order record
Data loss if the desktop fails	Firestore cloud sync means data survives even if the device doesn't
Scope creep into V2 features during V1 build	Lock V1 scope per Section 7; log all V2 requests separately

13. What Else the Restaurant Needs

- Reliable router, with the till desktop and (if used) printer LAN interface on stable network settings, plus WiFi coverage for the mobile device(s).
- Backup power (UPS or similar) — a power cut stops the till and the printer, since both run through the desktop.
- At least one staff phone (or a dedicated device) capable of running the mobile app.
- A designated admin (likely Enose) who can edit menu prices, void/reprint receipts, and view reports.
- Cash float management process — the app tracks recorded sales, not physical cash-in-drawer reconciliation, unless added to scope.
- Printed backup receipt book for the first week, as a safety net during parallel testing.

14. Starter Menu (seed data)

Item	Category	Suggested Price (KES)
Ugali	Sides	50
Kuku Fry (Chicken)	Main	350
Chips (Fries)	Sides	150
Sausages	Sides	100
Smokies	Sides	50
Mandazi	Snacks	20
Chapati	Sides	20
Beef Fry	Main	300
Beef Stew	Main	250

Placeholder prices — confirm actual prices with Enose before seeding. Also confirm whether combo items (e.g. "Ugali + Beef Stew") should exist as single tap-to-order items for speed.

15. Version 2 Roadmap (Post-Launch, Separately Scoped)

- AI Business Advisor — scheduled Cloud Function summarizing recent orders/expenses via Claude (Anthropic API).
- QR Coded Receipts — per-order QR code printed via ESC/POS QR command support.
- M-Pesa Sales Records — Safaricom Daraja API integration to reconcile STK Push confirmations against recorded orders.
- Automatic Inventory Deduction — ingredients/stock collection with a recipe map per menu item; low-stock alerts.
- User Accounts (Staff Logins) — Firebase Auth with role-based access (Owner/Admin vs. Staff/Cashier), enforced on both desktop and mobile.
- True Offline Mode — dedicated local DB (SQLite/Hive) with custom sync/conflict resolution for extended outages, on both platforms.
- Second Printer / Mobile Printing — direct mobile-to-printer printing (e.g. Bluetooth or a second LAN printer) if tableside receipt printing is ever wanted.

16. Appendix

- Printer: Xprinter XP-Q80A — USB + LAN, ESC/POS command support, 80mm paper.
- Windows raw printing reference: Win32 print spooler RAW mode, or a Flutter/Windows USB ESC/POS package.
- LAN fallback reference: raw TCP socket to the printer's IP on port 9100, using ESC/POS-formatted bytes.
- Firestore offline docs: confirm current persistence limits per platform (desktop and mobile) before relying on it for extended offline periods.